

Scene-CLIP: Enhancing Compositional Scene Understanding With Scene Graphs

Author information scrubbed for double-blind reviewing

No Institute Given

1 Appendix

1.1 Data Preparation

In this section, we describe the methodology for preparing scene-graph triples from captions in both the training and evaluation benchmark datasets. we utilize the FACTUAL scene graph parser to parse captions into scene-graph triples. To enhance accuracy, we leverage the “Discourse-Level” scene graph model, which enables iterative scene graph improvement through multi-round refinement. We prepare our datasets as Huggingface dataset objects and execute batched map operations across all “true captions” and “false captions” to create “true triples” and “false triples” respectively.

Datasets. For training, we use a combination of the COCO-2014 dataset augmented with crafted negatives and scene graph triples, and the recent VisMin training dataset. We select VisMin for its rigorous curation pipeline, which ensures minimal hard negatives, diverse visual and textual complexity, and, most importantly, human-validated captions. For evaluation, we consult the literature to identify the most critical benchmarks for assessing Scene-CLIP. First, we include Visual Genome (VG), which tests model sensitivity to simple word swaps—a known limitation highlighted in numerous studies. We also incorporate Winoground and two recent, sophisticated benchmarks: SugarCrepe and SugarCrepe++. These benchmarks focus on understanding fine-grained textual differences and address exploitable strategies that undermine previous evaluation methods. Furthermore, we report retrieval performance on the VisMin Benchmark, where our method demonstrates superior spatial reasoning capabilities compared to a fine-tuned ViT-L/14 CLIP model. To verify retention of the original model’s capabilities, we evaluate recall scores on the COCO test split. Comprehensive dataset configurations and sample statistics are detailed in Tables 1 and 2.

Handling failed captions. Some captions do not parse into triples successfully due to their brevity and structural ambiguity. Some examples of captions which are actual samples in evaluation benchmarks that fail to parse through the parser: “a trampolines are way to fun exercise”, “fun way to trampolines are a exercise”, “on a construction worker getting his moves”. Unfortunately, the scene-graph parser is not sufficiently sophisticated to handle such cases and returns an

Query: For each caption in the provided list of captions, extract at least one minimal triple in the format [Object, relation, object] or [Object, relation, attribute], selecting whichever format best fits the caption. Each triple must contain only the essential object, relation, and attribute/object words with no helper words, written in lowercase and enclosed in list format. For example, the caption “refrigerator on the right” should yield ['refrigerator','on','right']. Return a maximum of upto three triples per caption, in the same order as the captions provided.

Fig. 1. Prompt template followed to prompt the Gemini-2.5-pro model in cases where scene graph parsing failed or resulted in low-quality triples.

empty list of triples to indicate parsing failure. In other cases, the parser may produce a single low-quality triple, which gives the misleading impression of successful parsing simply because the returned list is non-empty. Both scenarios—empty lists and low-quality triples can adversely affect the contribution and quality of signals from the triple aggregator during both training and inference. To mitigate these issues, we employ a fallback processing mechanism using the Gemini-2.5 Pro model for problematic captions (see Figure 1 for the prompt template used). Once all samples contain at least one valid scene graph triple for both true and false captions, we mark the dataset as complete and proceed to use it for its designated purpose: training or evaluation. **Negative sampling strategy.** The negative sampling strategy used in our paper combines two distinct types of hard negatives: (1) *syntactic negatives* generated from the MS-COCO dataset by manipulating scene graph structures, and (2) *semantic hard negatives* sourced directly from the VisMin dataset. This two-pronged approach ensures the model learns to distinguish subtle variations across both structural and contextual dimensions. The complete process for constructing a training batch is formalized in Algorithm 1.

For captions from the MS-COCO dataset, we reuse the prepared dataset from Structure-CLIP since we found it to be well in line with the objective of including syntactic negatives by altering sentence structure by swapping objects and attributes. We outline the method that was used in preparing the scene graph triples for the COCO dataset below. Each caption is first converted into a scene graph $G(W_i)$, from which relational triples (O_1, R, O_2) and attribute pairs (A_1O_1) and (A_2O_2) are extracted. Guided by this scene graph, a high-quality

semantic negative sample W_i^- is generated by applying the sampling function

$$W_i^- = F(W_i, G(W_i)),$$

which constructs negatives that retain the overall scene composition while altering detailed semantics.

For relational triples of the form (O_1, R, O_2) , the semantic negative is produced by exchanging the subject and object:

$$W_i^- = \text{Swap}((O_1, R, O_2)) = (O_2, R, O_1),$$

thereby modifying the relational directionality without changing the participating elements.

For attribute pairs $(A_1 O_1)$ and $(A_2 O_2)$, attribute-level negatives are generated through

$$\begin{aligned} \text{Swap}((A_1 O_1), (A_2 O_2)) \\ = \begin{cases} (A_2 O_1), (A_1 O_2), & \text{if } O_1 \neq O_2, \\ \text{pass}, & \text{if } O_1 = O_2, \end{cases} \end{aligned}$$

ensuring that only attributes associated with different objects are exchanged. These structured modifications yield semantic negatives that maintain the same scene composition but differ in fine-grained relational or attribute semantics. The resulting altered triples are then used to form the negative caption paired with the original image during training.

The VisMin dataset provides expertly-curated image-caption pairs designed to test compositional understanding. Each positive sample, consisting of an image I_p and a caption C_p , is accompanied by a hard negative image I_n and caption C_n . The negative pair is created by applying a minimal, semantically meaningful edit (e.g., changing an object’s color, location, or count).

We leverage this structure by creating a symmetric training objective. For each sample from VisMin, we generate two training instances:

1. The original image I_p is paired with its true caption C_p (positive) and the hard negative caption C_n (negative).
2. The hard negative image I_n is paired with its true caption C_n (positive) and the original caption C_p (negative).

This approach doubles the effective training data from VisMin and provides a strong learning signal, teaching the model to become sensitive to the subtle compositional changes that differentiate the positive and negative pairs.

1.2 Training Setup

This section provides a complete description of the training configuration needed to reproduce our results, including all hyperparameters, architectural details of the triple aggregator, and the software as well as hardware environment.

Algorithm 1 Dual Negative Sampling Batch Construction

```

1: Input: A batch of data items  $\mathcal{B}$ 
2: Output: A processed batch  $\mathcal{B}_{\text{proc}}$ 
3:  $\mathcal{B}_{\text{proc}} \leftarrow \emptyset$ 
4: for all item  $b$  in  $\mathcal{B}$  do
5:   if  $b$  is from COCO then
6:      $(I, C) \leftarrow$  image and caption of  $b$ 
7:      $T_{\text{true}} \leftarrow \text{ParseSceneGraph}(C)$ 
8:     if a relational swap is possible in  $T_{\text{true}}$  then
9:        $T_{\text{false}} \leftarrow \text{RelationalSwap}(T_{\text{true}})$ 
10:    else if an attribute swap is possible in  $T_{\text{true}}$  then
11:       $T_{\text{false}} \leftarrow \text{AttributeSwap}(T_{\text{true}})$ 
12:    else
13:      continue
14:    end if
15:     $C_{\text{false}} \leftarrow \text{Caption}(T_{\text{false}})$ 
16:    Add  $(I, C, T_{\text{true}}, C_{\text{false}}, T_{\text{false}})$  to  $\mathcal{B}_{\text{proc}}$ 
17:  else if  $b$  is from VisMin then
18:     $(I_p, C_p, I_n, C_n) \leftarrow$  positive and negative pairs from  $b$ 
19:     $T_p \leftarrow \text{ParseSceneGraph}(C_p)$ 
20:     $T_n \leftarrow \text{ParseSceneGraph}(C_n)$ 
21:    Add  $(I_p, C_p, T_p, C_n, T_n)$  to  $\mathcal{B}_{\text{proc}}$ 
22:    Add  $(I_n, C_n, T_n, C_p, T_p)$  to  $\mathcal{B}_{\text{proc}}$ 
23:  end if
24: end for
25: return  $\mathcal{B}_{\text{proc}}$ 

```

Hardware and Environment. All models were implemented using PyTorch and the Hugging Face ‘transformers’ library. Training and evaluation were conducted on a single NVIDIA A100 GPU with 40GB of VRAM. We leveraged mixed-precision training (FP16) to optimize memory usage and accelerate computation. To ensure reproducibility, all experiments were run with fixed random seeds.

Hyperparameters. The detailed hyperparameter configuration can be found in Table 3.

Triple Aggregator Architecture. The triple aggregator described in our method is a lightweight, 2-layer transformer designed to process a variable-length sequence of scene graph triple embeddings and summarize them into a single, fixed-size structural feature vector. The architecture begins by prepending a special, learnable [CLS] token to the sequence of triple embeddings. This input sequence is then processed by a standard transformer encoder composed of $L = 2$ layers. Each layer features a multi-head self-attention block with $H = 8$ attention heads and a position-wise feed-forward network (FFN) that uses a GELU activation function. The FFN expands the model’s 512-dimensional embedding to a hidden dimension of 2048 before projecting it back. Following a Pre-LN de-

Table 1. Dataset statistics for training and evaluation.

Dataset	No. of Samples
<i>Training</i>	
MS-COCO	98,000
VisMin	112,000
MS-COCO + VisMin	200,000
<i>Evaluation</i>	
SugarCrepe	7,012
SugarCrepe++	4,757
VisMin Bench	2,084
VG Attribution	28,700
VG Relation	23,900
Winoground	400

sign, Layer Normalization is applied before each block, and residual connections are used throughout. To handle variable numbers of triples, an attention mask is employed to ignore padded inputs during the self-attention calculations. The final Knowledge Embedding is the output representation corresponding to the [CLS] token from the final layer. This entire module, which includes a dropout rate of 0.1 for regularization, is intentionally parameter-efficient, adding only approximately 2.1 million trainable parameters to the model.

1.3 Results and Discussion

Compositional robustness. While DAC-LLM achieves the top SugarCrepe score (86.4%), its performance drops sharply on SugarCrepe++ (45.3%), suggesting an over-reliance on lexical cues. In contrast, Scene-CLIP maintains high accuracy on both SugarCrepe (84.4%) and SugarCrepe++ (69.2%), setting a new state-of-the-art on the latter (Scene-DAC attains 69.7%). This resilience, particularly on difficult swap-based edits that test object-attribute binding, indicates that our model learns true compositional structure rather than surface-level correlations. Notably, our gains concentrate on the swap settings (Swap-Obj: 72.8%, Swap-Attr: 59.0%), evidencing robust object-attribute binding and role consistency under minimal edits.

Scene-CLIP (C+V) attains an average score of 82.9% on Visual Genome, matching or exceeding stronger baselines like Structure-CLIP while being parameter efficient. This supports our claim that a lightweight triple aggregator operating in CLIP space is sufficient for fine-grained grounding of structural information. Our model’s improved visual grounding is further evidenced by its high Image R@1 score of 14.8% on Winoground, demonstrating a strong ability to identify the correct image in ambiguous minimal-pair scenes by leveraging structured scene graph information.

On VisMin, Scene-CLIP (C+V) achieves an average group retrieval score of 68.6% and Scene-DAC obtaining 69.8%, both above Structure-CLIP (C+V)

Table 2. We show the Base datasets and their respective statistics used by various compositional enhancing methods.

Method	Base Dataset	No. of Samples
NegCLIP	MS-COCO (2014)	98K
DAC-LLM	CC3M	3M
DAC-SAM	CC3M	3M
SVLC-R	CC3M	–
SVLC-R+L	CC3M	–
CoN-CLIP	CC3M	188K
TripletCLIP	CC3M, CC12M	24M
CLIC-COCO	MS-COCO (2017)	118K
CLIC-LAION	LAION	1M
CLIC-CC12M	CC12M	1.2M
CLIC-RedCaps	PixelProse	850K
Structure-CLIP	MS-COCO	98K
Scene-CLIP	MS-COCO, VisMin	200K

Table 3. Hyperparameters used during training.

Hyperparameter	Value
Optimizer	AdamW
AdamW Betas	(0.9, 0.999)
Weight Decay	0.01
Learning Rate	5e-6
Batch Size	128
Training Epochs	10
Contrastive Loss Temperature τ	1 (fixed)
Margin Loss Margin m	0.2
Fusion Weight λ_f	0.2

which scores 68.0%. Category-wise, we are strongest on “Object” and “Counting”, and competitive on “Attribute”. Scene-DAC obtains superior “Relation” performance obtaining a group score of 68.3%, suggesting that higher-quality captions further amplify relational gains which is consistent with our thesis that structured supervision is most effective when captions capture precise relations. Refer to 4 for a side by side comparison of our method against Structure-CLIP.

No trade-off with general alignment. Unlike methods that specialize at the cost of retrieval (e.g., DAC variants on COCO), Scene-CLIP (C+V) preserves strong COCO R@1 scores. This suggests that structural features act as a compositional *regularizer* rather than replacing the caption’s global semantics. Our training data composition is crucial to this as outlined in the paper with an ablation study on the training datasets.

Why the design works. (1) *Data composition:* Training on COCO alone produces strong retrieval performance but fails on compositional tasks, while VisMin

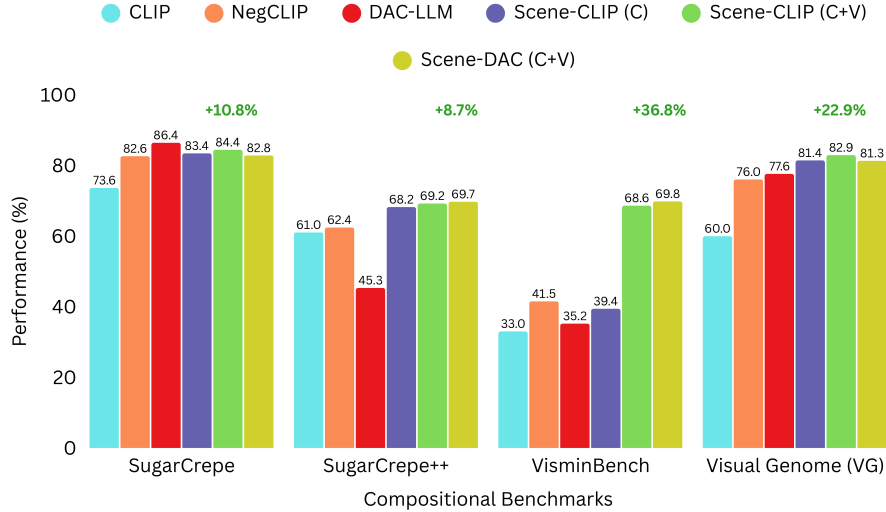


Fig. 2. Comparative performance of our method against a few baselines. We note the performance gain in % over CLIP in green for each benchmark. We report average accuracy scores across all categories for all benchmarks except VisMin where we report the overall group retrieval score.

alone improves spatial reasoning but hurts general performance. By combining them, COCO provides diverse training examples while VisMin provides targeted examples for learning compositional structures. (2) *Small fusion weight*: The optimal fusion weight ($\lambda = 0.2$) is small, confirming that structural features are most effective as a guiding signal. They inject crucial relational and attribute constraints without overpowering the contextual semantics of the global caption. Larger weights lead to a performance drop by over-emphasizing sparse triples at the expense of holistic meaning. (3) *Dual loss*: Our dual loss function operates through a powerful synergy. The contrastive loss aligns visual content with fused caption and structural features, teaching the model to recognize compositional structure. Simultaneously, the margin ranking loss strengthens the text encoder’s semantic precision by forcing it to distinguish true captions from difficult hard negatives. (4) *Train both encoders*: Freezing either the image or text encoder leads to a significant performance drop, particularly on VisMin. Learning fine-grained compositionality is not just a textual task and requires adapting the visual backbone to extract specific features needed to ground abstract relations and attributes.

Takeaways vs. baselines. Scene-CLIP is strongest where minimal edits demand correct role/attribute binding (SugarCrepe++, swaps), where object identity dominates retrieval (VisMin Object), and where cardinality matters (VisMin Counting). Structure-CLIP benefits from heavier modules but offers smaller gains per parameter. DAC-LLM shines on basic compositional tests but over-

Table 4. Scene-CLIP consistently outperforms Structure-CLIP across all major compositional benchmarks while using 28.4% fewer parameters, highlighting its superior architectural efficiency.

Benchmark	Scene-CLIP	Structure-CLIP	Δ (Absolute)
Parameters (M)	157.58	220.0	-28.4
SugarCrepe	84.4	82.2	+2.2
SugarCrepe++	69.7	67.8	+1.9
VisMin Group	69.8	68.0	+1.8
VG Avg.	82.9	82.6	+0.3

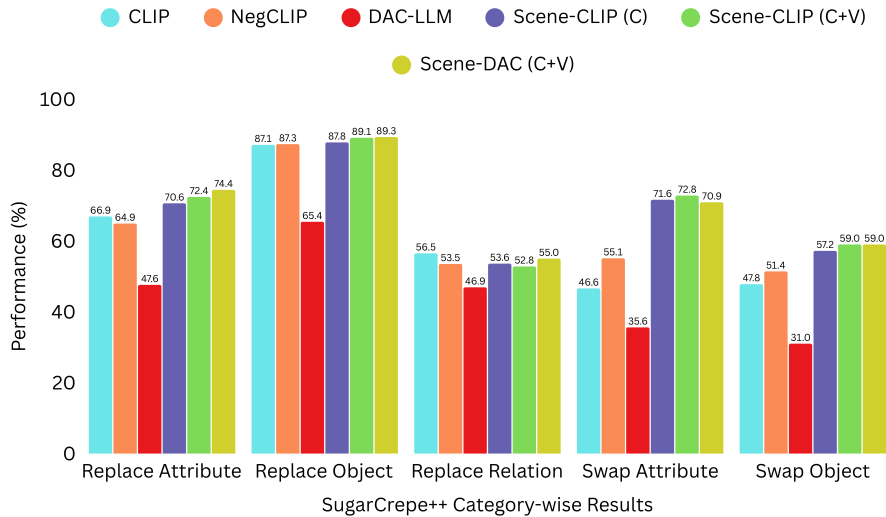


Fig. 3. Sugarcrepe++ category wise Image-Text task (ITT) accuracy results. Scene-DAC and Scene-CLIP variants demonstrate superior performance against baselines.

fits lexical signals. Scene-DAC shows that caption quality can further amplify relation retrieval, complementing our structural training signal.

Failure cases and limitations. Winoground text scores remain challenging, likely due to short, ambiguous prompts. Our approach depends on the quality of parsed triples; improving parsing or incorporating visual grounding into parsing could yield further gains. We analyze some cases where Scene-CLIP fails in 6. We also note that our model represents the inherent bias that is present in the MS-COCO and VisMin training datasets. We could not scale Scene-CLIP to hundreds of millions of pairs or larger models due to academic resource constraints.

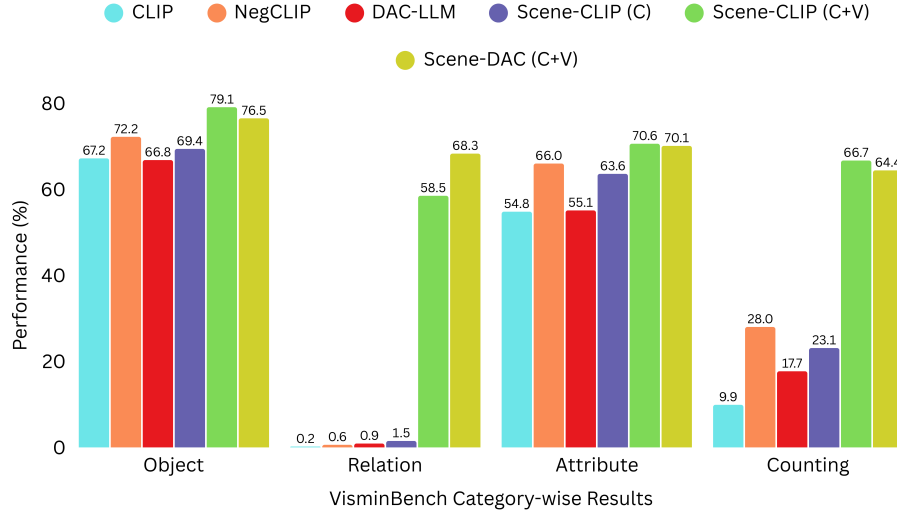


Fig. 4. VisminBench category wise group retrieval scores. Scene-DAC (C+V) and Scene-CLIP (C+V) demonstrate significant improvement in the “Relation” category while also demonstrating strong performance gains across the Object, Counting and Attribute tasks.

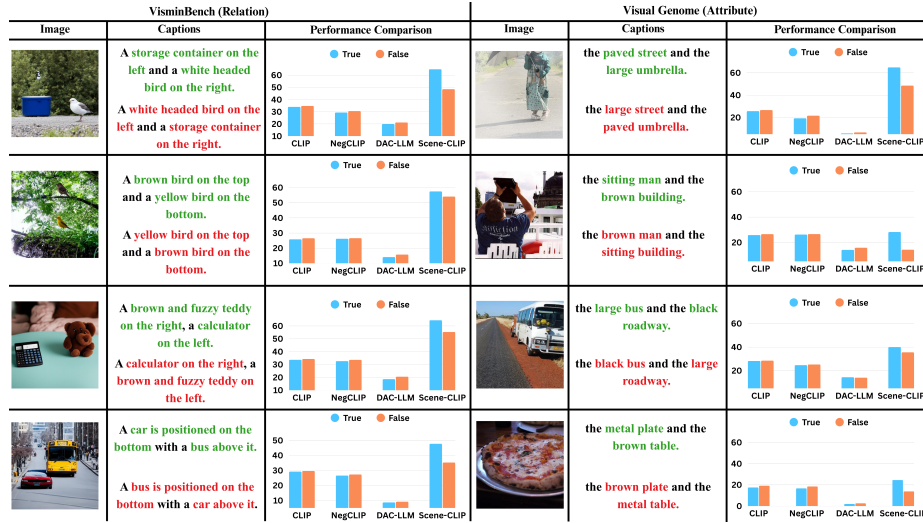


Fig. 5. Comparative analysis of CLIP scores (cosine-similarity) of Scene-CLIP against CLIP, NegCLIP and DAC-LLM on Vismin Benchmark’s Relation task and Visual Genome’s Attribute understanding task.

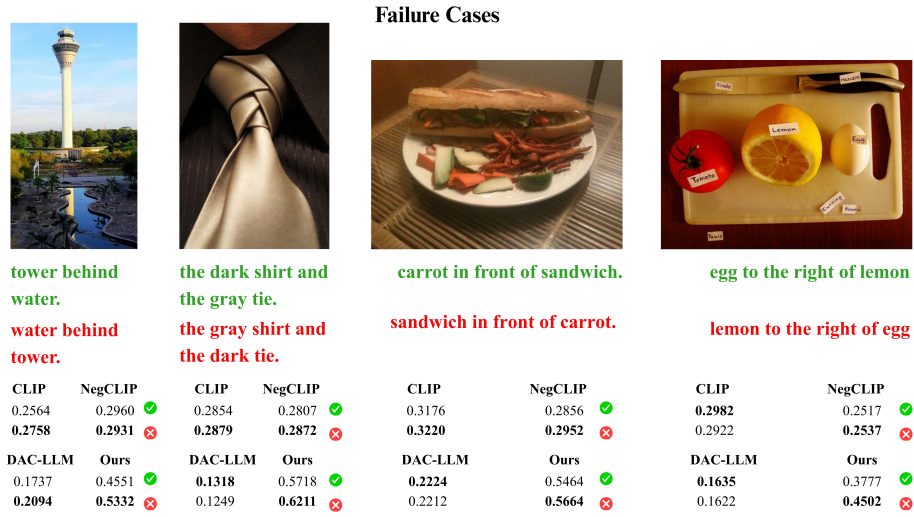


Fig. 6. Some failure cases of our model which demonstrate room for improvement.